

เครื่องมือที่ใช้สำหรับเขียนโปรแกรมเชิงวัตถุเบื้องต้น



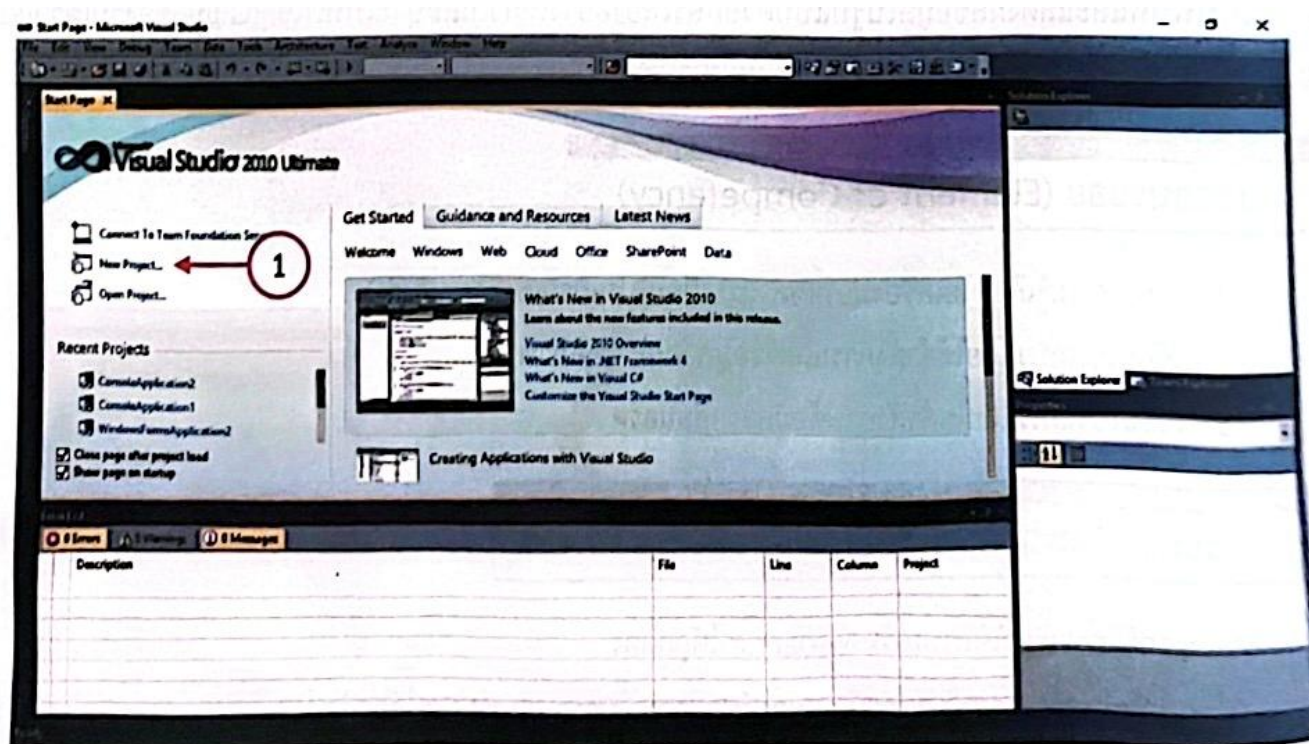
เนื้อหาสาระ (Content)

การพัฒนาโปรแกรมด้วยภาษา Visual C# จะประกอบด้วยขั้นตอนดังนี้ วิเคราะห์ปัญหาและความต้องการในการพัฒนาโปรแกรม เช่น โปรแกรมต้องการติดต่อกับผู้ใช้อย่างไร ข้อมูลที่ผู้ใช้จะป้อนให้กับโปรแกรมเป็นอย่างไร และผลลัพธ์ที่ต้องการถูกแสดงผลอย่างไร ออกแบบขั้นตอนวิธีด้วยการแสดงการทำงานของโปรแกรมออกมาเป็นภาพรวม แต่ละขั้นตอนมีความชัดเจนและสามารถเปลี่ยนให้อยู่ในรูปคำสั่งภาษา C# ได้ง่าย นำขั้นตอนวิธีที่ได้ ออกแบบไว้มาสร้างเป็นไฟล์โปรแกรม (Source Code) ที่ถูกต้องตามหลักไวยากรณ์ของภาษา C# แปลโปรแกรมให้อยู่ในรูปของภาษาเครื่องที่คอมพิวเตอร์เข้าใจและสามารถทำงานตามคำสั่งได้ ทดสอบการทำงานของโปรแกรม หากพบข้อผิดพลาดให้ตรวจสอบความถูกต้องในขั้นตอนที่ผ่านมา ซึ่งอาจหมายถึงการแก้ไข โปรแกรม ขั้นตอนวิธี หรือการวิเคราะห์ปัญหาและความต้องการใหม่

4.1 วิธีการเรียกใช้โปรแกรม Visual C#

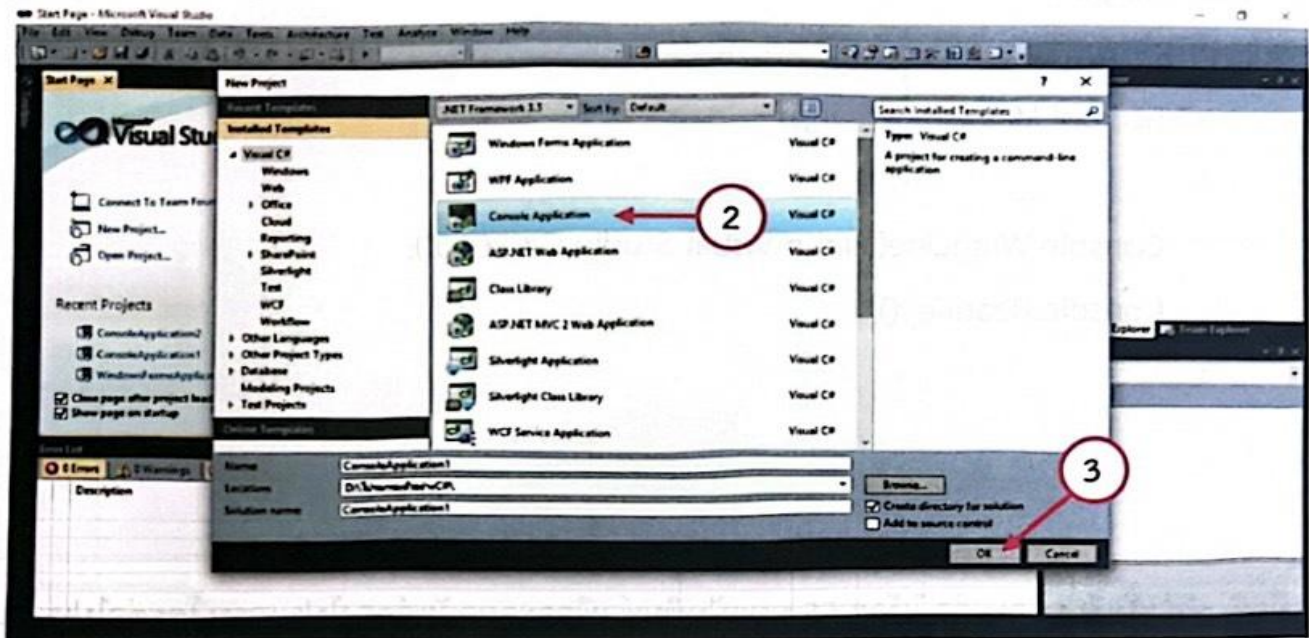
โปรแกรม Visual C# หลังจากติดตั้งโปรแกรมเสร็จแล้ว ก็จะสามารถเข้าใช้โปรแกรมได้ทันที โดยมีขั้นตอนดังนี้

เมื่อเข้าสู่โปรแกรม Visual Studio 2010 จะพบหน้าต่างดังนี้



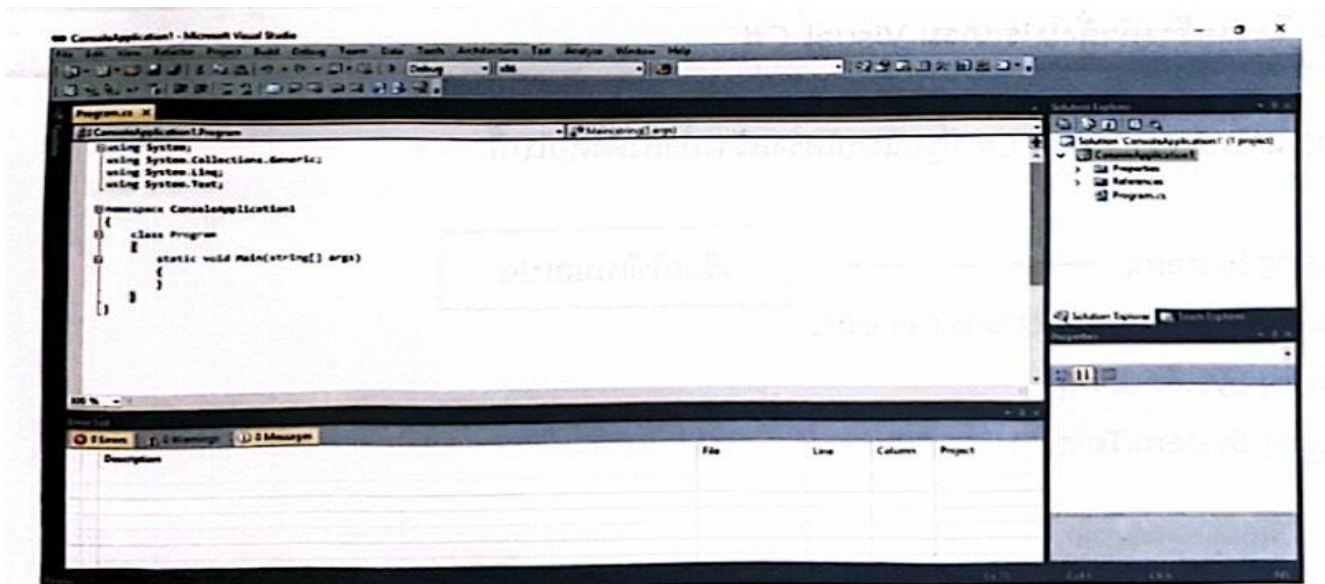
รูปที่ 4.1 เลือก New Project

การเขียนโปรแกรมนั้น จะต้องเริ่มจากการสร้างโปรเจกต์ (Project) ก่อน โดยการคลิกที่ลิงค์ New Project
หมายเลข 1 หรือคลิกที่เมนู File แล้วเลือก New Project ก็ได้ จากนั้น โปรแกรมจะเปิดหน้าต่าง
ขึ้นมา ให้เลือกรูปแบบของโปรแกรมที่ต้องการ ในที่นี้ให้เลือก Console Application
แล้วคลิกตกลง หมายเลข 2



รูปที่ 4.2 เลือก Console Application แล้วคลิก OK

จากนั้นจะได้ Project และไฟล์ที่ใช้ในการเขียนรหัสคำสั่งชื่อ Program.cs โดยไฟล์นี้โปรแกรม Visual C# จะสร้างรหัสโปรแกรมมาให้ส่วนหนึ่ง ทั้งนี้เพื่อช่วยให้สามารถเขียนโปรแกรมได้เร็วขึ้น ดังรูปที่ 4.3



รูปที่ 4.3 หน้าต่าง โปรแกรม Visual C#

จากนั้นให้ทดลองเขียนโปรแกรมตัวอย่างดังต่อไปนี้

```
namespace ConsoleApplication 1
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello Visual Studio 201010");

            Console.ReadKey();
        }
    }
}
```

เมื่อพิมพ์คำสั่งเรียบร้อยแล้ว ให้กด F5 บนแป้นพิมพ์ เพื่อจะดูผลลัพธ์ของโปรแกรม โดยต่อไปเราจะเรียกการเรียกดูผลลัพธ์ว่า การรันโปรแกรม (ใน Visual C# เรียกว่า Debugging) ผลลัพธ์ที่ได้จากการรันโปรแกรกดังนี้

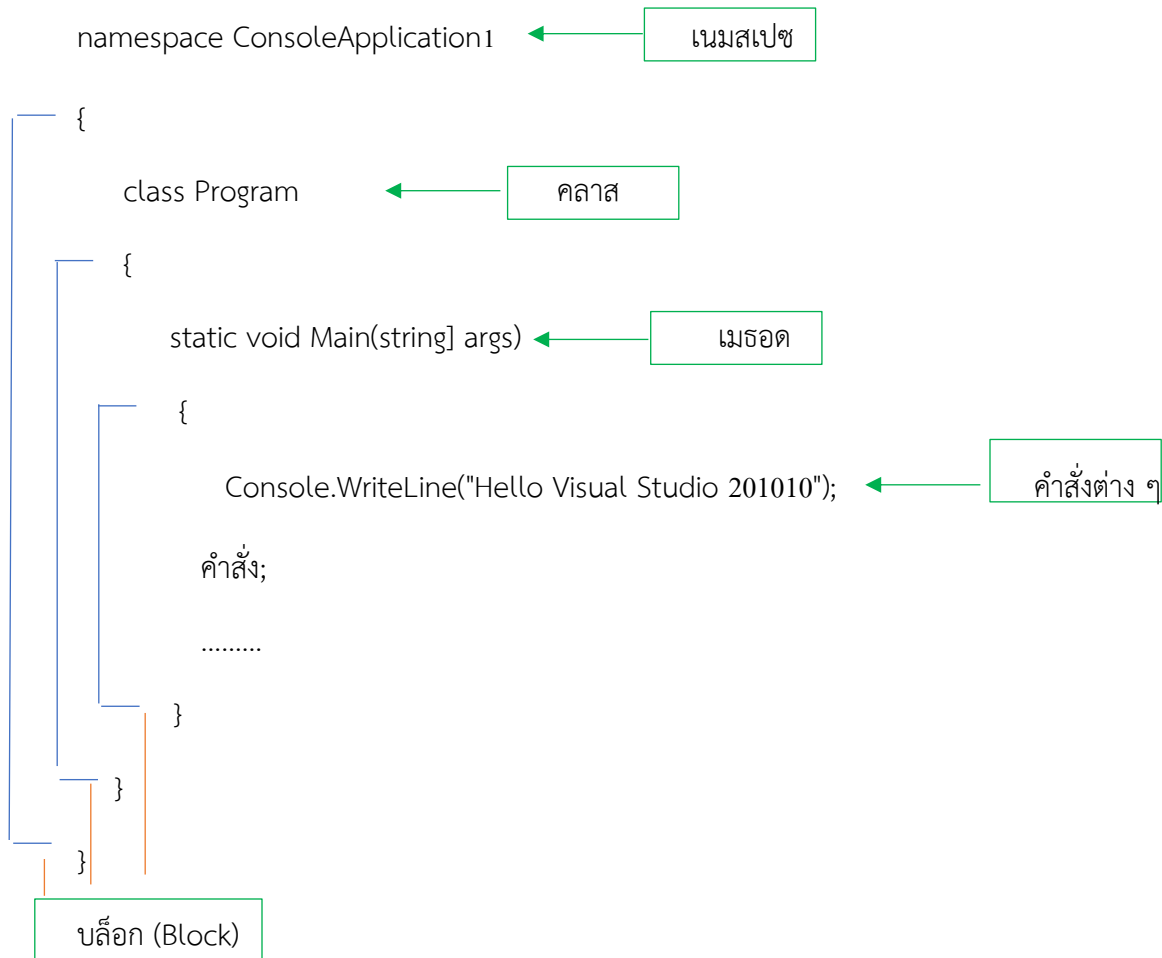
Hello Visual Studio C# 2010

4.2 โครงสร้างคำสั่งโปรแกรม Visual C#

โปรแกรมภาษา Visual C# มีรูปแบบโครงสร้างคำสั่งดังต่อไปนี้

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
```

← เรียกใช้เนมสเปซ



4.3 องค์ประกอบพื้นฐาน Visual C#

องค์ประกอบพื้นฐานที่สำคัญในการเขียนโปรแกรม Visual C# ที่ควรรู้มีดังต่อไปนี้

4.3.1 บล็อก {...}

รูปแบบของโปรแกรม Visual C# จะใช้บล็อก {...} ในการกำหนดจุดเริ่มต้นและจุดสิ้นสุดของการทำงานในแต่ละส่วนของโปรแกรม ซึ่งภายในแต่ละบล็อกอาจมีบล็อกย่อย ๆ ซ้อนลงไปได้อีก ตามลักษณะของงาน เช่น

```

Class className
{
    If (.....)
    {
        For (.....)
        {
            .....
            .....
        }
        .....
        .....
    }
}

```

โดยที่เครื่องหมาย "{" (open brace) กับ "}" (close brace) นั้นจะต้องควบคู่กันพอดี และโดยทั่วไปแล้วเราจะนิยามวง "{" ไว้ให้ตรงกับคำสั่งเริ่มต้นบล็อก เช่น จากรหัสคำสั่งตัวอย่างที่เราได้วางไว้ตรงกับคำสั่ง if หรือ for เป็นต้น ซึ่งทั้งนี้เพื่อให้สามารถพิจารณาขอบเขตของบล็อกได้ง่ายขึ้น แต่ทั้งนี้ไม่ใช่กฎข้อบังคับแต่อย่างใด ซึ่งเราอาจเลือกวางในแบบที่เราถนัดก็ได้

4.3.2 เครื่องหมายสิ้นสุดคำสั่ง (;)

ในโปรแกรม Visual C# เราจะใช้เครื่องหมายเซมิโคลอน (;) เป็นตัวแสดงจุดสิ้นสุดของแต่ละคำสั่ง หากเราไม่ใส่เครื่องหมายนี้เพื่อคั่นระหว่างคำสั่งแล้ว โปรแกรมจะถือว่าเป็นคำสั่งเดียวกันไปตลอด ถึงแม้ว่าจะอยู่คนละบรรทัดก็ตาม เช่น

```

X = 20;
Y = "XXX";
Z = x +
    20;

```

เมื่อเครื่องหมาย ; เป็นตัวบอกว่าเป็นจุดสิ้นสุดของคำสั่ง คำสั่งต่าง ๆ จึงจะสามารถมาอยู่ในบรรทัดเดียวกันได้ เช่น

```
X=20;y="XXX";Z=X+20;
```

แต่ในการเขียนรหัสคำสั่งในลักษณะนี้ จะไม่นิยมเขียน เนื่องจากอ่านโปรแกรมได้ยาก และรหัสโปรแกรมดูไม่เป็นระเบียบ แต่อาจนำมาใช้ในบางกรณีได้

บางคำสั่งเราไม่สามารถทำให้จบภายในบรรทัดเดียวกันได้ เพราะคำสั่งอาจมีคำจะมีคำสั่งย่อย ๆ อยู่ภายในอีกก็ได้ ดังนั้น คำสั่งเหล่านี้จะมีบล็อกของตนเอง เช่น คำสั่ง if คำสั่ง for คำสั่ง while เป็นต้น เมื่อใดที่มีบล็อกอยู่แล้วก็ไม่จำเป็นจะต้องใส่เครื่องหมาย ; ต่อท้ายเครื่องหมายบล็อกอีก แต่ถ้าหากคำสั่งภายใน ต้องมีเครื่องหมาย ; ตามปกติ เช่น

```
if (เงื่อนไข)
```

```
{
```

```
    คำสั่ง A;
```

```
    คำสั่ง B;
```

```
}
```

นั่นหมายถึง ก็คือ จะไม่มี ; อยู่ทั้งก่อนหน้า “{“ และหลัง “}”

4.3.3 การเขียนคำอธิบายประกอบในรหัสโปรแกรม

คำอธิบาย (Comment) หมายถึง การเขียนข้อความใด ๆ ที่ไม่ใช่คำสั่งแต่เขียนปะปนกันไปกับคำสั่งอื่น ๆ เพื่ออธิบายเรื่องใดเรื่องหนึ่งเอาไว้เพื่อความเข้าใจในรหัสโปรแกรมตรงส่วนนั้น ๆ โดยทั้งนี้เพื่อให้ โปรแกรมไม่สับสนว่าส่วนใดเป็นคำสั่ง ส่วนใดเป็นแค่เพียงคำอธิบาย ซึ่งไม่เกี่ยวข้องกับการทำงานของโปรแกรม จึงต้องใช้เครื่องหมายมาเป็นตัวช่วยในการแยกแยะ การแทรกคำอธิบาย โดยสามารถทำได้ 2 ลักษณะดังนี้ คือ

1. รูปแบบ // คำอธิบาย

จะใช้สำหรับอธิบายบรรทัดเดียว (Single Line comment) โดยโปรแกรมจะถือว่าตั้งแต่สัญลักษณ์ // เป็นต้นไปจนถึงสิ้นสุดบรรทัดจะเป็นคำอธิบายทั้งหมด จะไม่นำมาพิจารณาในการประมวลผล ของโปรแกรม เช่น

```
// สูตรการคำนวณหาขนาดพื้นที่สี่เหลี่ยมผืนผ้า
```

```
RectangleArea=width*length;
```

หรือ

```
RectangleArea=width*length; //สูตรการคำนวณหาขนาดพื้นที่สี่เหลี่ยมผืนผ้า
```

1. รูปแบบ /* คำอธิบาย */

ในกรณีที่ คำอธิบายโปรแกรมของเราค่อนข้างยาว จำเป็นต้องเขียนหลาย ๆ บรรทัด การใช้ // หลาย ๆ ครั้ง จึงอาจไม่สะดวกนัก เราสามารถใช้ /*.. */ ครอบคำอธิบายนั้นแทนได้ ซึ่งโปรแกรมจะถือว่า /* เป็นต้นไปจะเป็นคำอธิบายทั้งหมด จนกว่าจะเจอสัญลักษณ์ */ จึงจะถือว่าเป็นการสิ้นสุดคำอธิบาย เช่น

```
/* นี่เป็นส่วนของคำอธิบาย
```

```
ที่ไม่มีผลต่อการทำงานของโปรแกรม
```

```
มีไว้เพื่อช่วยให้ทำความเข้าใจในการเขียนรหัสคำสั่งได้ง่ายขึ้น
```



การเขียนโปรแกรมในโหมด Console Application ด้วย Visual C#

ในการเขียนโปรแกรมในโหมด Console Application นั้น โปรแกรม Visual C# ได้เตรียมคำสั่งพื้นฐานสำหรับการแสดงผลข้อมูลออกทางจอภาพเพื่อใช้ในการตรวจสอบผลลัพธ์จากการทำงานของโปรแกรม ดังนี้

4.4.1 คำสั่งการแสดงผลข้อมูลออกทางจอภาพ แบ่งออกเป็น 2 คำสั่ง ดังต่อไปนี้

1. คำสั่ง Write()

รูปแบบคำสั่ง

`Console.Write(อักขระ/สตริง/ตัวเลข/ตัวแปร/นิพจน์/เมธอด);`

คำอธิบาย การแสดงผลข้อมูลสามารถแสดงได้หลายรูปแบบดังต่อไปนี้

(1) **อักขระ** หมายถึง ตัวอักขระทั่ว ๆ ไป โดยตัวอักขระจะต้องอยู่ภายใต้เครื่องหมาย '....' และในเครื่องหมายนี้จะมีอักขระได้เพียง 1 ตัวอักขระเท่านั้น เช่น

```
Console.Write('A');
```

(2) **สตริง** หมายถึง ประโยคข้อความต่าง ๆ โดยสตริงจะต้องอยู่ภายใต้เครื่องหมาย '....' และจะมีอักขระก็ได้ เช่น

```
Console.Write("Hello");
```

(3) **ตัวเลข** หมายถึง ตัวเลขจำนวนเต็มหรือทศนิยมใด ๆ และไม่ต้องอยู่ภายใต้เครื่องหมายใด ๆ เช่น

```
Console.Write(100);
```

(4) **ตัวแปร** หมายถึง ตัวแปรทุกตัวในโปรแกรม ไม่ว่าจะเป็นตัวแปรชนิดสตริง ตัวเลข หรือบูลีน เช่น

```
int age = 50;
```

```
Console.Write(age);
```

(5) **นิพจน์** หมายถึง นิพจน์ต่าง ๆ เช่น นิพจน์ทางคณิตศาสตร์ นิพจน์เปรียบเทียบ เป็นต้น เช่น

```
int age = 50;
```

```
Console.Write(age/2); //นิพจน์ทางคณิตศาสตร์
```

```
Console.Write(age>๓๐); //นิพจน์เปรียบเทียบ
```

(6) **เมธอด** หมายถึง ทั้งเมธอดสำเร็จและเมธอดที่เราสร้างขึ้นที่มีการส่งค่ากลับ เช่น

```
Console.Write(Math.sqrt(9));
```


2. คำสั่ง WriteLine()

รูปแบบคำสั่ง

```
Console.WriteLine(อักขระ/สตริง/ตัวเลข/ตัวแปร/นิพจน์/เมธอด);
```

จะเห็นว่าคำสั่ง WriteLine() สามารถแสดงข้อมูลได้เหมือนกับคำสั่ง Console. Write() แต่แตกต่างกันที่ผลลัพธ์ในการแสดงผล ดังตัวอย่างต่อไปนี้

```
Console.WriteLine(1);
```

```
Console.Write(2);
```

```
Console.Write(3);
```

ผลลัพธ์ 123

```
Console.WriteLine(1);
```

```
Console.WriteLine(2);
```

```
Console.WriteLine(3);
```

ผลลัพธ์ 1

2

3

คำสั่ง `Write()` และ `WriteLine()` จะสามารถแสดงข้อมูลได้หลายลักษณะ รวมทั้งสามารถกำหนดรูปแบบของการแสดงผลของตัวเลข และวันที่ได้อย่างหลากหลาย ดังตัวอย่างต่อไป

รูปแบบการแสดงผลพื้นฐาน

```
Console.WriteLine("Hello"); // "Hello"
```

```
Console.WriteLine("100"); // "100"
```

```
Console.WriteLine(100+100);           //"200"
```

```
Console.WriteLine("Hello"+"300");           //"Hello300"
```

```
Console.WriteLine("300"+"300");           //"300300"
```

รูปแบบการแสดงผลโดยใช้เครื่องหมายบล็อก {...} แทนตำแหน่งที่วางข้อมูลซึ่งใส่เลขลำดับ
เริ่มจาก 0

```
Console.WriteLine("{0},"Hello"); // "Hello"
```

```
Console.WriteLine("{0}",50); // "50"
```

```

Console.WriteLine("abcdefg{0}", "Hello");           //"abcdefgHello"

Console.WriteLine("Result= {0}", 50);               //"Result= 50"

Console.WriteLine(" {0} ---- {1}", 50, 100);        //"50 ---100"

Console.WriteLine("Width={0} Length={1} Area={2}", 10, 20, 10*20);

                                                    //"width=10 Length=20 Area=200"

```

ตัวอย่างการแสดงผลของเลขจำนวนเต็ม

```

Console.WriteLine("{0:00000}", 15);                //"00015"

Console.WriteLine("{0:00000}", -15);                //" -00015"

Console.WriteLine("{0,5}, 15");                    //" 15"

Console.WriteLine("{0, -5}, 15");                   //"15 "

Console.WriteLine("{0,5:000}, 15 ");                //" 015"

Console.WriteLine("{0, -5:000}", 15);               //"015 "

Console.WriteLine("{0:#,minus #}", 15);             //"15"

Console.WriteLine("{0:#,minus #}", -15);            //"minus 15"

Console.WriteLine("{0:#,minus #,zero}", 0);         //"zero"

Console.WriteLine("{0:+### ### ##}", 447900123456);

                                                    //" +447900123456"

Console.WriteLine("{0:+ ##-###-###}", 8958712551);  //"89-5871-2551"

```

ตัวอย่างการแสดงผลของเลขจำนวนเต็ม

```

Console.WriteLine("{0:0.00}, 123.4567);           //"123.46"

Console.WriteLine("{0:0.00}, 123.4);               //"123.40"

Console.WriteLine("{0:0.00}, 123.0);               //"123.00"

Console.WriteLine("{0:0.##}, 123.4567);            //"123.46"

Console.WriteLine("{0:0.##}, 123.4);               //"123.4"

Console.WriteLine("{0:0.##}, 123.0);               //"123"

Console.WriteLine("{0:00.0}, 123.4567);            //"123.5"

Console.WriteLine("{0:00.0}, 23.4567);             //"23.5"

```

```

Console.WriteLine("{0:00.0},3.4567);    //"03.5"
Console.WriteLine("{0:00.0},-3.4567);    //" -03.5"
Console.WriteLine("{0:0.0.0},12345.67);    //"12,345.7"
Console.WriteLine("{0:0,0},12345.67);    //"12,346"
Console.WriteLine("{0:0.0},0.0);    //" -0.0"

```

ตัวอย่างการแสดงผลวันที่

```

//create date time 2019-07-18 14:29

DateTime dt=new DateTime(2019,7,18,14,29);

Console.WriteLine("{0:t}",dt);           //"4:05 PM"           ShortTime
Console.WriteLine("{0:d}",dt);           //"3/9/2008"           ShortDate
Console.WriteLine("{0:T}",dt);           //" 4:05:07 PM"        LongTime
Console.WriteLine("{0:d}",dt);           //"Sunday,March 09,2008" LongDate

```

4.4.2 คำสั่งอ่านข้อมูลจากคีย์บอร์ด

เป็นคำสั่งสำหรับอ่านหรือรับข้อมูลจากคีย์บอร์ด ณ จุดที่รับคำสั่งหรือจุดที่เคอร์เซอร์รออยู่ มี 2 รูปแบบคือ

1. Console.Read() เป็นการอ่านค่าข้อมูลในรูปแบบของ Integer (int) ดังตัวอย่าง

```

using System;

using System.Collections.Generic;

using System.Text;

namespace ConsoleApplication1
{
    class Program
    {
        static void Main(string[] args)
        {
            int i = Console.Read();

        }
    }
}

```

2. Console.ReadLine() เป็นการอ่านค่าข้อมูลในรูปแบบของ string ดังตัวอย่าง

```
using System;

using System.Collections.Generic;

using System.Text;

namespace ConsoleApplication1
{
    class Program
    {
        static void Main(string[] args)
        {
            string str = Console.ReadLine();
        }
    }
}
```

ตัวอย่างที่ 1

```
using System;

using System.Collections.Generic;

using System.Text;

namespace ConsoleApplication1
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.Read();
        }
    }
}
```

ผลลัพธ์โปรแกรม

Empty program _

ตัวอย่างที่ 2

```
using System;

using System.Collections.Generic;

using System.Text;

namespace ConsoleApplication1
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Programming in C# is easy.");

            Console.Read();
        }
    }
}
```

ผลลัพธ์โปรแกรม

Programming in C# is easy._

ตัวอย่างที่ 3

```
using System;

using System.Collections.Generic;

using System.Text;

namespace ConsoleApplication1
{
    class Program
    {
        static void Main(string[] args)
        {
```



```
        Console.WriteLine("Programming in C# is easy.");

        Console.Read();

    }

}

}
```

ผลลัพธ์โปรแกรม

Programming in C# is easy.

-

ตัวอย่างที่ 4 โปรแกรมที่มีการกำหนดค่าให้กับตัวแปร

```
using System;

using System.Collections.Generic;

using System.Text;

namespace ConsoleApplication1
{
    class Program
    {
        static void Main(string[] args)
        {
            int sum;

            sum = 500 + 15;

            Console.WriteLine("The sum of 500 and 15 is " + sum);

            Console.Read();

        }

    }

}
```

ผลลัพธ์โปรแกรม

The sum of 500 and 15 is 515

ตัวอย่างที่ 5

```
using System;

using System.Collections.Generic;

using System.Text;

namespace ConsoleApplication1
{
    class Program
    {
        static void Main(string[] args)
        {
            int sum, value;

            sum = 10; value = 15;

            Console.WriteLine(sum + "\t" + value);

            Console.Read();
        }
    }
}
```

ผลลัพธ์โปรแกรม

10 15

-

สรุปสาระสำคัญ

ในการเขียนโปรแกรมในโหมด Console Application ด้วย Visual C# นั้น จะต้องเข้าใจถึงโครงสร้างคำสั่งโปรแกรม Visual C#

การเขียนโปรแกรมภาษา Visual C# นั้น มีองค์ประกอบพื้นฐานที่สำคัญในการเขียนโปรแกรม Visual C# ที่ควรรู้มีดังต่อไปนี้

บล็อก {...}

รูปแบบของโปรแกรม Visual C# จะใช้บล็อก {...} ในการกำหนดจุดเริ่มต้นและจุดสิ้นสุดของการทำงานในแต่ละส่วนของโปรแกรม ซึ่งภายในแต่ละบล็อกอาจมีบล็อกย่อย ๆ ซ้อนลงไปได้อีก ตามลักษณะของงาน เช่น

```
Class className
```

```
{  
  
    If (.....)  
  
    {  
  
        For (.....)  
  
        {  
  
            .....  
  
            .....  
  
        }  
  
        .....  
  
        .....  
  
    }  
  
}
```

โดยที่เครื่องหมาย "{" (open brace) กับ "}" (close brace) นั้นจะต้องควบคู่กันพอดี และโดยทั่วไปแล้วเราจะนิยมนำ "{" ไว้ให้ตรงกับคำสั่งเริ่มต้นบล็อก เช่น จากระทัดคำสั่งตัวอย่างที่เราได้วางไว้ตรงตรงตรงกับคำสั่ง if หรือ for เป็นต้น ซึ่งทั้งนี้เพื่อให้สามารถพิจารณาขอบเขตของบล็อกได้ง่ายขึ้น แต่ทั้งนี้ไม่ใช่กฎข้อบังคับแต่อย่างใด ซึ่งเราอาจเลือกวางในแบบที่เราถนัดก็ได้

เครื่องหมายสิ้นสุดคำสั่ง (;)

ในโปรแกรม Visual C# เราจะใช้เครื่องหมายเซมิโคลอน (;) เป็นตัวแสดงจุดสิ้นสุดของแต่ละคำสั่ง หากเราไม่ใส่เครื่องหมายนี้เพื่อคั่นระหว่างคำสั่งแล้ว โปรแกรมจะถือว่าเป็นคำสั่งเดียวกันไปตลอด ถึงแม้ว่าจะอยู่คนละบรรทัดก็ตาม

การเขียนคำอธิบายประกอบในรหัสโปรแกรม

สามารถทำได้ 2 ลักษณะดังนี้ คือ

1. รูปแบบ // คำอธิบาย

ใช้สำหรับอธิบายบรรทัดเดียว (Single Line comment) โดยโปรแกรมจะถือว่าตั้งแต่สัญลักษณ์ // เป็นต้นไปจนถึงสิ้นสุดบรรทัดจะเป็นคำอธิบายทั้งหมด แต่ไม่นำมาพิจารณาในการประมวลผลของโปรแกรม เช่น

// สูตรการคำนวณหาขนาดพื้นที่สี่เหลี่ยมผืนผ้า

RectangleArea=width*length;

หรือ

RectangleArea=width*length; //สูตรการคำนวณหาขนาดพื้นที่สี่เหลี่ยมผืนผ้า

2. รูปแบบ /* คำอธิบาย */

ในกรณีที่คำอธิบายโปรแกรมของเราค่อนข้างยาว จำเป็นต้องเขียนหลายๆ บรรทัด การใช้ // หลาย ๆ ครั้ง จึงอาจไม่สะดวกนัก เราสามารถใช้ /*...*/ ครอบคำอธิบายนั้นแทนได้ ซึ่งโปรแกรมจะถือว่า /*เป็นต้นไปจะเป็นคำอธิบายทั้งหมด จนกว่าจะเจอสัญลักษณ์ */ จึงจะถือว่าเป็นการสิ้นสุดคำอธิบาย เช่น

/* นี่เป็นส่วน of คำอธิบาย

ที่ไม่มีผลต่อการทำงานของโปรแกรม

มีไว้เพื่อช่วยให้ทำความเข้าใจในการเขียนรหัสคำสั่งได้ง่ายขึ้น */

คำสั่งในการการแสดงผลข้อมูลออกทางจอภาพ ซึ่งเป็นคำสั่งพื้นฐาน

1. คำสั่ง Write()

รูปแบบคำสั่ง

Console.Write(อักขระ/สตริง/ตัวเลข/ตัวแปร/นิพจน์/เมธอด);

2. คำสั่ง WriteLine()

รูปแบบคำสั่ง

Console.WriteLine(อักขระ/สตริง/ตัวเลข/ตัวแปร/นิพจน์/เมธอด);

คำสั่งอ่านข้อมูลจากคีย์บอร์ด

เป็นคำสั่งสำหรับอ่านหรือรับข้อมูลจากคีย์บอร์ด ณ จุดที่รับคำสั่งหรือจุดที่เคอร์เซอร์รออยู่ มี 2 รูปแบบคือ

1. Console.Read() เป็นการอ่านค่าข้อมูลในรูปแบบของ Integer (int)

2. Console.ReadLine() เป็นการอ่านค่าข้อมูลในรูปแบบของ string